Winning Strategy: Build an “Attention Engine,” Not a Watchlist

Your current ideas are directionally strong, but the biggest opportunity is to avoid building a dashboard that merely displays price, volume, news, and sentiment. The winning product should answer one question extremely well:

> **“I was away from the market. What changed in my watchlist, why does it matter, and what should I look at first?”**

That positioning directly addresses the challenge’s wording: users should understand what has “meaningfully changed” since they last checked and what deserves attention now.

A polished, opinionated product with a clear prioritisation model will likely perform better than a feature-heavy but generic terminal.

1. Product Thesis

I would frame the product as:

> **Pulse: Your watchlist’s change briefing**

Instead of showing every update equally, Pulse:

1. Captures a user’s last meaningful checkpoint.
2. Detects material changes across price, volatility, volume, news, sentiment, and events.
3. Groups related signals into a single “story.”
4. Ranks stories by importance.
5. Explains them in plain language.
6. Lets the user mark them as reviewed.
7. Preserves an audit trail so the next visit starts from the correct state.

A user returning after three days should not see the same list of stocks. They should see:

```
You have 4 things worth reviewing

1. Reliance Industries — High attention
   Price rose 5.8% in 72 hours with 2.4× normal volume.
   A board announcement was published yesterday.

2. Tata Motors — Medium attention
```

```
   Price declined 3.1%, while sector peers were broadly flat.

3. Infosys — Low attention
   Negative news sentiment increased, but price movement remains muted.

No meaningful change:
HDFC Bank, ITC
```

That is substantially more useful than:

```
Reliance ₹2,845 +5.8%
Volume 2.4×
News: Positive
```

2. What Counts as a Meaningful Change?

Do not define meaningful change as one universal threshold. A 3% move may be significant for a large-cap stock but ordinary for a highly volatile small-cap stock.

Use a combination of:

- Absolute threshold.
- Historical baseline.
- Relative movement versus peers or the index.
- Confidence in the underlying data.
- Whether multiple signals agree.
- Whether the event is new since the user's last visit.

Recommended signal categories

A. Price movement

Track returns over multiple periods:

- Since last viewed.
- 1 day.
- 3 trading days.
- 1 week.
- 1 month.
- 3 months.
- 6 months.
- 1 year.

The important addition is **since last viewed**. The challenge is specifically about what changed after the user left, not only standard market periods.

Use both absolute and relative return:

$$\text{Absolute return} = \frac{P_t - P_0}{P_0} \times 100$$

$$\text{Relative return} = \text{Stock return} - \text{Benchmark return}$$

For example:

```
Tata Motors is down 4.2% since your last visit,
underperforming Nifty Auto by 2.9 percentage points.
```

This is more meaningful than simply showing "-4.2%."

B. Volatility expansion

A price move becomes more important when volatility is unusually high.

Possible MVP metric:

$$\text{Volatility expansion} = \frac{\text{Recent 5-day volatility}}{\text{Previous 30-day volatility}}$$

Example:

```
Volatility increased 1.8× versus its recent baseline.
```

You do not need to implement complex options analytics. A simple rolling standard deviation of daily returns is sufficient.

C. Volume anomaly

Your volume-spike idea is valuable, but avoid writing "institutional buying" as a fact. High volume can result from:

- Institutional activity.
- Retail participation.
- News.
- Block deals.
- Corporate actions.
- Forced buying or selling.
- Intraday speculation.

Use neutral language:

> “Trading volume is 2.6× the recent average.”

A useful baseline is:

$$\text{Volume ratio} = \frac{\text{Current period volume}}{\text{Average volume over previous } n \text{ periods}}$$

Recommended baselines:

- Intraday: compare the same time-of-day volume over the past 20 sessions.
- Daily: compare with the previous 20 trading days.
- Weekly: compare with the previous 8–12 weeks.

A simple rule:

```
Volume anomaly = current volume / 20-day average volume
```

Suggested interpretation:

Volume ratio	Interpretation
Below 1.25×	Normal
1.25×–1.75×	Elevated
1.75×–2.5×	Significant
Above 2.5×	Exceptional

Pair volume with price direction:

Price	Volume	Suggested explanation
Up	High	Strong upward participation
Down	High	Strong selling participation
Up	Low	Weak or unconfirmed move
Down	Low	Weak or potentially low-conviction decline

This creates a better user-facing insight:

```
Price rose 4.1% with 2.8× normal volume — a high-participation move.
```

D. Relative performance

A stock can be up while still underperforming its sector.

Compare each stock against:

- Nifty 50.

- Relevant sector index.
- A user-selected benchmark.
- Optional peer basket.

Example:

```
Infosys gained 1.2%, but underperformed the Nifty IT index by 3.4 percentage points.
```

This is a high-value differentiator because many basic watchlists do not contextualise returns.

E. News and filing events

This should be more than “news available.” Classify events into categories:

- Earnings or results.
- Guidance changes.
- Management changes.
- Board meeting.
- Fundraising or dilution.
- Merger or acquisition.
- Regulatory action.
- Credit rating.
- Contract or order win.
- Dividend, bonus, split, or record date.
- Promoter buying, selling, or pledge.
- Litigation.
- Plant shutdown or operational disruption.
- Analyst or institutional meeting.
- General corporate update.

The NSE corporate-filings page exposes company, subject, details, attachment, and broadcast time, which makes exchange filings a suitable foundation for an event timeline. cite [1]

For the MVP, do not attempt unrestricted AI news interpretation. Build a deterministic event classifier first:

```
Subject contains:
- “Outcome of Board Meeting” → Board/event
- “Credit Rating” → Credit risk
- “Change in Director” → Management
- “Record Date” → Corporate action
- “Analyst/Institutional Investor Meet” → Investor communication
- “Acquisition” or “Merger” → Strategic event
```

Then optionally use an LLM to create a concise explanation from the filing text.

F. Sentiment shift

Sentiment should not be presented as absolute truth. Use it as a supporting signal.

Track:

- Current sentiment.
- Previous sentiment.
- Number of articles.
- Source quality.
- Sentiment direction.
- Confidence.

A useful representation:

```
Sentiment shifted from neutral to negative across 8 articles.
Price has not yet moved materially.
```

That is far more valuable than:

```
Sentiment: Negative
```

Use three dimensions:

```
Sentiment score: -0.42
Change: -0.31 versus previous period
Coverage: 8 articles
```

Important: distinguish “no news” from “neutral news.”

- No news: no meaningful coverage detected.
- Neutral news: coverage exists but is not directionally strong.
- Unknown: data unavailable or stale.

G. Fundamental or business metrics

For a 72-hour project, do not try to implement a full fundamental analysis platform. Instead, surface only event-driven fundamental changes:

- Latest earnings announcement.
- Revenue or profit surprise, if available.
- Guidance change.
- Margin expansion or contraction.

- Debt or credit-rating update.
- Promoter holding change.

If you do include fundamental data, clearly label its date:

```
Last reported quarterly result: 43 days ago
```

This prevents stale information from appearing current.

3. The Core Ranking Model

The best feature in the project should be the **Attention Score**.

Do not make users manually interpret ten columns. Rank each stock and explain the ranking.

Suggested attention score

Create a score from 0 to 100:

$$A = w_p P + w_v V + w_r R + w_e E + w_s S + w_n N - w_q Q$$

Where:

- P: price movement score.
- V: volume anomaly score.
- R: relative-performance score.
- E: event significance score.
- S: sentiment-shift score.
- N: news novelty or coverage score.
- Q: data-quality penalty.

For an MVP, use:

Component	Weight
Price and volatility	30%
Volume anomaly	20%
Relative performance	15%
Corporate event	20%
Sentiment shift	10%
Data confidence and freshness	5%

Do not make the score mysterious. Show the reasons:

```
Attention score: 84

Why it is high:
- Price moved +5.6% since last visit.
- Volume is 2.7× its 20-day average.
- A board announcement appeared today.
- Stock outperformed its sector by 3.1%.
```

Signal agreement

A powerful improvement is to reward multiple independent signals agreeing.

For example:

```
Price up + volume high + positive event = high confidence
```

But:

```
Price up + no volume confirmation + no event = moderate confidence
```

You can represent this as:

```
Signal confidence: High
Evidence: 4 independent signals
```

This is more credible than using an AI-generated label such as "Strong Buy," which would create both trust and regulatory concerns.

Avoid buy/sell language

Do not show:

- Buy.
- Sell.
- Strong buy.
- Target price.
- Guaranteed opportunity.
- Institutional buying detected.

Use:

- High attention.
- Review recommended.
- Unusual activity.
- Positive price-volume alignment.

- Event requires review.
- Data is inconclusive.

The product should help users understand information, not issue investment recommendations. In India, specific-stock recommendations and buy/sell calls can fall into regulated investment-advice territory, so the product should be framed as an information and monitoring tool rather than an advice engine. cite [2]

4. The Most Important Concept: Checkpoints

The requirement “return later and see what has changed” needs a proper state model.

Avoid simply comparing against the previous API fetch. That creates bad behavior because background refreshes would constantly move the baseline.

Instead, maintain two concepts:

A. Last seen

When the user opens a watchlist, record:

```
watchlist_last_seen_at
```

This is the point from which the next session calculates changes.

B. Last acknowledged

When the user clicks “Mark reviewed,” record per-stock acknowledgement:

```
stock_last_acknowledged_at
```

This lets users manage attention independently.

Example:

- User opens the app at 9:00 AM.
- Reliance has a new announcement.
- User does not open its detail drawer.
- The event remains “unreviewed.”
- User returns at 1:00 PM.
- The event is still highlighted.

That is better than marking everything read merely because the watchlist page loaded.

Recommended data model

```
users
- id
- created_at

watchlists
- id
- user_id
- name
- created_at

watchlist_items
- id
- watchlist_id
- instrument_id
- position
- created_at
- removed_at

watchlist_preferences
- watchlist_id
- price_threshold_pct
- volume_threshold_ratio
- sentiment_enabled
- event_alerts_enabled
- benchmark_symbol

user_checkpoints
- user_id
- watchlist_id
- last_opened_at
- last_reviewed_at

instrument_snapshots
- instrument_id
- captured_at
- price
- volume
- volatility
- sentiment_score
- sentiment_count

events
- id
- instrument_id
- event_type
- title
- summary
- source_url
- published_at
- detected_at
- confidence

user_event_state
- user_id
```

```
- event_id
- seen_at
- acknowledged_at
- dismissed_at
```

For a mock-up, you can simplify this into a JSON or SQLite schema, but demonstrate that your design supports persistence across sessions and devices.

5. The User Experience I Would Build

Screen 1: Watchlist briefing

This should be the default landing page.

Header

```
Good morning, Prateek

Your watchlist changed since yesterday
Last checked: Yesterday, 6:42 PM
```

Summary cards

```
4 need attention
2 major price moves
1 new corporate event
3 unusual volume signals
```

Main sections

Priority feed

```
Needs your attention

Reliance Industries
+5.8% since last visit
Volume 2.4× normal
New board announcement
High attention

Tata Motors
-3.2% since last visit
Underperformed sector by 2.1%
Negative sentiment shift
Medium attention
```

Quiet stocks

```
No meaningful change

HDFC Bank
ITC
Asian Paints
```

This prevents the user from scanning every row.

Screen 2: Timeline mode

Each stock should have a timeline:

```
Today, 10:34 AM
Price +2.7%; volume reached 2.1× average

Today, 9:45 AM
NSE filing: Outcome of board meeting

Yesterday
Sentiment shifted from neutral to positive

3 days ago
Price was ₹1,240.50
```

This creates a strong "what happened while I was away?" experience.

Screen 3: Explain-this-change drawer

When clicking a stock:

```
Why this is highlighted

Price
₹1,240 → ₹1,310
+5.65% since your last visit

Participation
Today's volume is 2.4× the 20-day average

Relative performance
Outperformed Nifty 50 by 3.2 percentage points

New information
Board-meeting outcome published 2 hours ago

Sentiment
Positive coverage increased from 42% to 68%
Based on 11 articles

Data freshness
```

```
Market data: 2 minutes ago
Event data: 18 minutes ago
```

The important design principle is that every alert must answer:

1. What changed?
2. Compared with what?
3. Why should I care?
4. How fresh is the information?
5. Where did it come from?

Screen 4: Add stock flow

Support:

- Search by symbol or company name.
- Add to an existing watchlist.
- Create a new watchlist.
- Remove or reorder stocks.
- Prevent duplicates.
- Show whether the instrument is NSE or BSE.
- Display confirmation after adding.

Example:

```
Search “Tata”
Tata Motors — NSE
Tata Steel — NSE
Tata Consumer Products — NSE
```

Screen 5: Alert preferences

Avoid exposing too many technical controls at first.

Use presets:

```
Attention sensitivity
○ Relaxed
● Balanced
○ High

Notify me when:
☑ Price moves materially
☑ Volume becomes unusual
☑ A company event is published
☑ Sentiment shifts sharply
```

Then provide advanced controls:

```
Price threshold: 3%
Volume threshold: 2× normal
Minimum event confidence: Medium
Benchmark: Nifty 50
```

6. Improve Your Proposed Ideas

Your ideas are good, but I would modify them as follows.

Your idea	Keep?	Improvement
Price delta since last seen	Yes	Add absolute, percentage, and benchmark-relative change
Configurable threshold notification	Yes	Add per-watchlist sensitivity and event deduplication
Volume spikes	Yes	Say "unusual volume," not "institutional buying"
Day, 72 hours, week, month, 4m, 6m, year	Partly	Use 1D, 3D, 1W, 1M, 3M, 6M, 1Y; treat 72 hours as 3 trading days
Sentiment shift	Yes	Show source count, direction, confidence, and time window
AI-extracted company events	Yes	Ground them in official filings and show source links
Notifications	Yes	Build in-app notifications first; simulate push/email if needed
Complex fundamental analysis	No for MVP	Add only result and guidance events

The biggest correction is **72 hours**. Markets are not open continuously, so "72 hours" can mean different numbers of trading sessions depending on weekends and holidays. Label it as:

```
Since last visit
3 trading days
```

You can still provide a 72-hour timestamp-based comparison, but the UI should make the distinction clear.

7. Technical Scope for 72 Hours

Do not attempt to build a production-scale data platform. Build a believable vertical slice with real or realistically seeded data.

Recommended stack

Frontend

- Next.js.
- TypeScript.
- Tailwind CSS.
- Recharts or lightweight SVG charts.
- TanStack Query for server state.
- Zustand or local state for UI state.

Backend

- Next.js API routes or FastAPI.
- PostgreSQL or SQLite.
- Prisma if using TypeScript.
- A scheduled ingestion script.
- Server-side scoring function.

Deployment

- Vercel for frontend/API.
- Supabase or Neon for PostgreSQL.
- Optional Redis only if genuinely needed.

For a 72-hour challenge, a single deployable application is preferable to multiple microservices.

Data strategy

You have three practical options:

Option 1: Real Groww API

Groww's API documentation exposes live quote data containing fields such as price, day change, OHLC, volume, market capitalisation, 52-week range, and market-depth values. Its historical data API provides candles with timestamp, OHLC, and volume across intervals such as 1 minute, 5 minutes, 1 hour, 4 hours, daily, and weekly. cite [3] [4]

This is ideal if you have credentials and permission to use the data.

Option 2: Seeded snapshot data

Create a realistic dataset for 15–30 popular NSE stocks with:

- Current price.
- Previous checkpoint price.
- Historical candles.

- Volume baselines.
- Events.
- Sentiment records.

This is often safer for a demo because the experience remains deterministic.

Option 3: Hybrid

Use real market prices but seed events and sentiment. Clearly label data freshness.

For the judging demo, deterministic data is usually better than an API that sometimes fails.

Suggested initial universe

Use a small, recognisable set:

- Reliance Industries.
- HDFC Bank.
- ICICI Bank.
- Infosys.
- TCS.
- Tata Motors.
- Tata Steel.
- ITC.
- Bharti Airtel.
- Asian Paints.
- Zomato.
- Bajaj Finance.

A 12-stock dataset is enough to demonstrate meaningful ranking.

8. Data Pipeline

The pipeline can be simple:

```
Market quote source
        ↓
Historical candle fetcher
        ↓
Corporate event fetcher
        ↓
News/sentiment processor
        ↓
Snapshot store
        ↓
Change detector
```

```
        ↓
Attention score
        ↓
Watchlist briefing API
        ↓
Frontend
```

Ingestion jobs

Market data job

Run periodically:

```
GET latest price and quote
GET historical candles
calculate returns
calculate volume ratios
calculate volatility
store snapshot
```

Events job

Fetch or seed filings:

```
instrument
event type
title
source
published time
summary
confidence
```

Sentiment job

For a prototype, precompute sentiment scores rather than trying to build a robust NLP system from scratch.

Example:

```
{
  "symbol": "RELIANCE",
  "sentiment": 0.58,
  "previous_sentiment": 0.18,
  "article_count": 9,
  "window": "72h",
  "confidence": 0.81
}
```

Scoring job

The scoring function should be deterministic and testable.

```
type Signal = {
  type: "price" | "volume" | "relative" | "event" | "sentiment";
  score: number;
  message: string;
  confidence: "low" | "medium" | "high";
};

function calculateAttentionScore(signals: Signal[]) {
  return Math.min(
    100,
    signals.reduce((total, signal) => total + signal.score, 0)
  );
}
```

9. Avoid Alert Fatigue

This is a major product-quality issue.

If a stock moves 1% every five minutes, do not create a new alert every time.

Use alert lifecycle states:

```
Detected → Updated → Acknowledged → Resolved
```

Group related changes into one story:

```
Reliance momentum story

- Price +5.8%
- Volume 2.4×
- Board event published
- Positive sentiment shift
```

Do not show four separate cards for the same underlying event.

Deduplication rules

- Same event title and stock: one event.
- Same story updated within a time window: update existing card.
- Repeated price threshold crossing: notify once until the stock returns below the threshold.
- Sentiment change requires a minimum article count.
- Do not generate sentiment alerts when coverage is too low.

Example:

```
Sentiment shift is not shown if fewer than 3 credible articles are available.
```

10. Handle Stale, Delayed, and Conflicting Data

This is explicitly mentioned in the challenge, so demonstrate that you thought about it.

Show freshness

Every data group should have its own timestamp:

```
Price: updated 2 min ago
Volume: updated 2 min ago
News: updated 18 min ago
Exchange filing: published 34 min ago
Sentiment: calculated 25 min ago
```

Do not use one generic “last updated” value for everything.

Stale-data policy

```
Fresh: less than 5 minutes old
Delayed: 5-30 minutes old
Stale: more than 30 minutes old
Unavailable: no valid data
```

The exact thresholds can vary by data type.

If a quote is stale:

```
Price data delayed by 42 minutes.
Attention score may be incomplete.
```

Conflicting data

If two sources disagree:

```
Price discrepancy detected
Source A: ₹1,242.10
Source B: ₹1,241.80
Difference: 0.02%
Using Source A because it is fresher.
```

For a prototype, you can implement a simple source-priority rule:

1. Official or licensed source.
2. Most recent timestamp.

3. Source with the smallest discrepancy.
4. Otherwise mark as conflicting.

Market closed state

Do not make a stock look inactive when the market is closed.

Show:

```
Market closed
Last traded at 3:30 PM IST
Changes shown against the previous trading session
```

Also account for Indian market holidays and weekends.

11. What to Build First

Must-have

1. Create, rename, and delete watchlists.
2. Add and remove stocks.
3. Persist watchlists across reloads.
4. Capture last-seen checkpoint.
5. Show "since last visit" changes.
6. Rank stocks by attention score.
7. Display clear reasons behind each score.
8. Show a detail timeline.
9. Include at least one event signal.
10. Include volume anomaly.
11. Include data freshness.
12. Provide a demo reset or seeded scenario.

Strong differentiators

1. Group signals into stories.
2. Compare performance against Nifty or sector.
3. "Mark reviewed" behavior.
4. Attention score explanation.
5. Quiet-state section showing what did not change.
6. Event-source links.
7. Configurable thresholds.

8. Responsive mobile layout.
9. Empty, loading, stale, and error states.
10. A guided demo mode.

Avoid

- Full brokerage order placement.
- Portfolio management.
- Complex technical indicators.
- Options chains.
- Chatbot-first UX.
- Excessive charting.
- Generic AI summaries with no sources.
- Buy/sell recommendations.
- An overengineered microservice architecture.
- Real-time websockets unless necessary for the core demo.

12. Suggested 72-Hour Plan

Hours 0–6: Product and data foundation

- Finalise the product thesis.
- Define signal types.
- Define attention score.
- Create schema.
- Seed or connect data.
- Prepare a demo scenario.
- Sketch the main screens.

Deliverable:

```
A user can return after a checkpoint and receive a ranked change briefing.
```

Hours 6–18: Backend and scoring

Build:

- Watchlist CRUD.
- Instrument search.
- Snapshot model.

- Checkpoint model.
- Change calculations.
- Attention scoring.
- Event model.
- Briefing endpoint.

Example endpoint:

```
GET /api/watchlists/:id/briefing
```

Response:

```
{
  "lastSeenAt": "2026-09-03T15:30:00+05:30",
  "summary": {
    "itemsNeedingAttention": 4,
    "priceMoves": 2,
    "volumeAnomalies": 3,
    "newEvents": 1
  },
  "stories": []
}
```

Hours 18–34: Main frontend

Build:

- Sidebar/watchlist navigation.
- Briefing header.
- Summary cards.
- Ranked story feed.
- Stock row/card.
- Add-stock flow.
- Loading and empty states.

Hours 34–46: Detail and explanation

Build:

- Stock detail drawer.
- Timeline.
- Mini chart.
- Signal breakdown.
- Freshness labels.

- Source links.
- Mark reviewed.
- Dismiss event.

Hours 46–58: Polish and resilience

Implement:

- Threshold settings.
- Responsive design.
- Error states.
- Stale-data banner.
- Market-closed state.
- Alert grouping.
- Sorting and filtering.
- Demo reset.

Hours 58–66: Testing and deployment

Test:

- First-time user.
- Returning user.
- No changes.
- Many changes.
- Duplicate stock.
- Removed stock.
- Stale data.
- Event acknowledgement.
- Mobile viewport.
- Browser refresh.
- API failure.

Hours 66–72: Demo and presentation

Prepare:

- Seeded demo account.
- A 2-minute walkthrough.
- Architecture diagram.
- Product rationale.

- Known limitations.
- Future roadmap.
- README with setup instructions.

Do not spend the final hours adding random features. Spend them improving reliability, visual hierarchy, and narrative clarity.

13. Demo Scenario for Judges

You should control the story shown during the demo.

Initial state

```
Watchlist: My Long-Term Stocks

Last checked:
Yesterday at 3:45 PM
```

On return

The app shows:

```
5 meaningful changes since your last visit
```

Story 1: High attention

```
Reliance Industries
+5.8% since last visit
Volume 2.4× normal
New board-meeting outcome
Positive sentiment shift
```

Story 2: Medium attention

```
Tata Motors
-3.4% since last visit
Underperformed Nifty Auto by 2.2%
No new official event
```

Story 3: Information-only

```
Infosys
New investor-meeting announcement
Price unchanged
```

Quiet section

```
No meaningful change:
HDFC Bank
ITC
Asian Paints
```

Then demonstrate:

1. Click Reliance.
2. See the explanation.
3. Open the official event source.
4. Mark it reviewed.
5. Return later.
6. Show that only unresolved changes remain.
7. Change the sensitivity setting.
8. Add a stock to the watchlist.
9. Refresh the page and show persistence.

This demonstrates almost every requirement without needing a large feature set.

14. Visual Design Direction

Groww is known for a relatively accessible, consumer-friendly experience. Do not make this look like a dense Bloomberg terminal.

Design principles

- One primary action: understand what changed.
- Use cards for stories, not every metric.
- Use colour sparingly.
- Green and red should communicate direction, not urgency alone.
- Use amber for unusual attention.
- Use neutral grey for stale or unavailable data.
- Make explanations readable before making charts detailed.
- Use progressive disclosure: summary first, evidence second.

Suggested attention colors

- Green: positive movement.
- Red: negative movement.
- Amber: unusual activity.
- Blue: new information.
- Grey: unchanged or unavailable.

Do not use red and green as the only indicators. Add labels and icons for accessibility:

```
↑ +5.8%
Unusual volume
New event
```

15. Metrics You Can Mention in Your Presentation

Even if you do not have real users, define success metrics.

Core product metrics

- Time to understand the first meaningful change.
- Percentage of sessions where a user opens a highlighted story.
- Percentage of alerts acknowledged.
- False-alert rate.
- Number of unnecessary repeated alerts.
- Watchlist retention.
- Time since last visit to first action.
- Percentage of users who can correctly explain why a stock was highlighted.

Quality metrics

- Data freshness.
- Event classification accuracy.
- Sentiment confidence.
- Percentage of stories with at least two supporting signals.
- Percentage of alerts with a source.
- API error rate.

A particularly strong metric is:

> **Signal-to-noise ratio: meaningful stories divided by total surfaced updates.**

That directly reflects the challenge.

16. Architecture Explanation for Judges

Use this simple architecture:

```
                ┌───────────────────┐
                │   Web Frontend    │
                │ Next.js + React   │
                └─────────┬─────────┘
                          │
                ┌─────────▼─────────┐
                │     API Layer     │
                │ Watchlists        │
                │ Briefings         │
                │ Preferences       │
                └─────────┬─────────┘
                          │
         ┌────────────────▼─────────────────┐
         │          Change Engine           │
         │ Price, volume, relative return   │
         │ Events, sentiment, freshness     │
         └─────────┬───────────┬────────────┘
                   │           │
         ┌─────────▼────┐ ┌────▼────────────┐
         │ Market Data  │ │ Events/News     │
         │ Quotes/OHLCV │ │ Filings         │
         └──────────────┘ └─────────────────┘
                          │
                  ┌───────▼──────────┐
                  │  PostgreSQL      │
                  │ Snapshots        │
                  │ Checkpoints      │
                  │ Watchlists       │
                  └──────────────────┘
```

Explain the architectural decisions:

- Snapshot data is persisted so changes can be computed reliably.
- The frontend does not calculate financial signals.
- The scoring engine produces explainable outputs.
- Event summaries retain source references.
- User checkpoints are separate from data-refresh timestamps.
- The system can later move ingestion to queues without changing the product API.

17. A Strong API Contract

Briefing endpoint

```
GET /api/watchlists/{watchlistId}/briefing
```

```
{
  "watchlist": {
    "id": "wl_123",
    "name": "My Stocks",
    "lastSeenAt": "2026-09-03T15:45:00+05:30"
  },
  "summary": {
    "attentionCount": 4,
    "highAttentionCount": 1,
    "newEventCount": 1,
    "volumeAnomalyCount": 3
  },
  "stories": [
    {
      "instrument": {
        "symbol": "RELIANCE",
        "name": "Reliance Industries",
        "exchange": "NSE"
      },
      "attentionScore": 87,
      "severity": "high",
      "changeState": "new",
      "signals": [
        {
          "type": "price",
          "label": "Price move",
          "value": "+5.8%",
          "description": "Since your last visit"
        },
        {
          "type": "volume",
          "label": "Unusual volume",
          "value": "2.4×",
          "description": "Compared with 20-day average"
        },
        {
          "type": "event",
          "label": "New company event",
          "value": "Board meeting outcome",
          "source": "NSE"
        }
      ],
      "freshness": {
        "price": "2026-09-04T15:27:00+05:30",
        "volume": "2026-09-04T15:27:00+05:30",
        "event": "2026-09-04T14:58:00+05:30"
      }
```

```
    }
  ]
}
```

The key is that the API returns **explanations**, not merely raw metrics.

18. The One Feature I Would Name and Own

Give the product a memorable feature:

"Since You Left"

This can be the main mode of the application.

```
Since You Left
4 meaningful changes
Last checked yesterday at 3:45 PM
```

Each card can show:

```
What changed
Why it matters
Evidence
Source
Action
```

This is much more memorable than calling it "Smart Watchlist."

Other possible names:

- Pulse.
- Market Brief.
- Watchlist Radar.
- Signalboard.
- Market Memory.
- What Changed?

My preference would be:

> **Groww Pulse — Since You Left**

It is easy to explain and directly maps to the challenge.

19. Final Recommendation

Build a narrow but excellent product around these five ideas:

1. **Checkpoint-based comparison** rather than generic daily returns.
2. **Attention ranking** instead of a flat watchlist.
3. **Grouped stories** instead of separate noisy alerts.
4. **Explainable evidence** instead of black-box AI scores.
5. **Freshness and source transparency** instead of pretending all data is equally reliable.

Your proposed metrics should become supporting evidence inside an attention story:

```
Reliance needs review because:
- It rose 5.8% since your last visit.
- Volume is 2.4× its normal level.
- It outperformed the benchmark by 3.1%.
- A new exchange filing was published.
- News sentiment shifted positive.
```

That is the difference between a technically competent watchlist and a product judges remember.

The strongest 72-hour submission is not the one with the most indicators. It is the one where a judge can immediately understand:

> **"I was away for two days, opened this product, and instantly knew what changed and why it deserved my attention."**

⁂

1. https://www.nseindia.com/companies-listing/corporate-filings-announcements
2. https://brandsaathi.ai/blog/sebi-finfluencer-rules-social-media
3. https://groww.in/trade-api/docs/curl/live-data
4. https://groww.in/trade-api/docs/curl/historical-data
5. https://groww.in/
6. https://github.com/topics/groww?o=asc&s=updated
7. https://groww.in/trade-api/docs/python-sdk/feed
8. https://www.truedata.in/
9. https://github.com/SAGAR-TAMANG/groww-python-mcp/blob/main/README.md
10. https://groww.in/trade-api/docs/python-sdk
11. https://groww.in/stocks/screens
12. https://help.trendspider.com/kb/scanner/market-scanner
13. https://libraries.io/pypi/pyGrowwAPI
14. https://nsearchives.nseindia.com/s3fs-public/inline-files/Download 15 mins delayed data tariff.pdf

15. https://www.nseindia.com/static/investor-relations/corporate-actions
16. https://nsearchives.nseindia.com/web/sites/default/files/inline-files/Download15mindelayeddatatariff.pdf
17. https://www.sebi.gov.in/social_media_rel_disclaimer.html
18. https://www.sebi.gov.in/sebi_data/attachdocs/jun-2025/sb_recommendation_interim_order.pdf
19. https://www.nse.in/market-data/stocks-traded
20. https://www.nseindia.com/static/market-data/real-time-data-subscription
21. https://www.nse.in/companies-listing/corporate-filings-announcements-xbrl-trading-window-disclosure
22. https://www.nseindia.com/companies-listing/corporate-filings-announcements-xbrl
23. https://github.com/bshada/nse-bse-api
24. https://bennythadikaran.github.io/NseIndiaApi/api.html
25. https://parse.bot/marketplace/2a81eefa-a201-41f8-af3b-103a9010df01/nseindia-com-api
26. https://skamalj.github.io/nseindiapy/api/corporate/
27. image.jpg
28. https://groww.in/markets/volume-shockers
29. https://github.com/RelativelyBurberry/Indian-Stock-News-Sentiment-Analysis
30. https://groww.in/trade-api/docs/python-sdk/live-data
31. https://groww.in/trade-api